

CryptoTalk

Master Developer Prompt — Bug Fixes & Feature Additions

*Paste this entire document into your AI coding assistant (Cursor, Gemini, Copilot, or Claude) along with your full project source code.
Follow the numbered sections in order.*

SECTION 1 — Project Context (Read This First)

You are working on CryptoTalk, a secure Android messaging app built with:

- Kotlin + Jetpack Compose (UI)
- Firebase Firestore (real-time database) + Firebase Auth (Google Sign-In)
- Hybrid Encryption: AES-256-GCM for message content, RSA-2048 for key wrapping
- Android Keystore for hardware-backed private key storage
- MVVM architecture — ViewModels, Repositories, Compose UI

Key files to understand before making any changes:

File	Role
<code>crypto/MessageCrypto.kt</code>	encrypt() and decrypt() — the core hybrid encryption engine
<code>crypto/KeystoreHelper.kt</code>	RSA keypair generation and retrieval from Android Keystore
<code>data/repository/MessageRepository.kt</code>	Sends/receives messages; calls encryption before saving to Firestore
<code>data/repository/UserRepository.kt</code>	Manages user public keys in Firestore
<code>data/repository/AuthRepository.kt</code>	Google Sign-In, sign-out, session management
<code>ui/chat/ChatScreen.kt</code>	Main chat UI — message bubbles, text input, send button
<code>ui/chat/ChatViewModel.kt</code>	Chat screen logic — state, send, delete, edit actions
<code>ui/conversationlist/ConversationListScreen.kt</code>	Home screen listing all conversations
<code>ui/newconversation/NewConversationViewModel.kt</code>	Find user by email, start new chat
<code>navigation/NavRoutes.kt</code>	Screen route definitions
<code>CryptoTalkApplication.kt</code>	App-level singletons — repositories initialized here

SECTION 2 — Critical Bug Fixes (Do These First)

These are the three bugs that will crash the app or break encryption. Fix them in the order listed.

BUG FIX #1 — Private Key Lost on Reinstall / New Device [SEVERITY: CRITICAL]

Problem

The RSA private key is generated inside Android Keystore on first launch. If the user reinstalls the app, clears app data, or switches phones, the Keystore is wiped. The private key is gone permanently. Every past message in Firestore becomes unreadable. The user also cannot receive any new messages because senders will encrypt to the OLD public key that no longer has a matching private key.

What to do in KeystoreHelper.kt

Add a function to check whether a valid keypair currently exists:

```
// In KeystoreHelper.kt - add this function
fun isValidKeyPair(alias: String = "cryptotalk_rsa_key"): Boolean {
    return try {
        val keyStore = KeyStore.getInstance("AndroidKeyStore")
        keyStore.load(null)
        keyStore.containsAlias(alias) &&
            keyStore.getEntry(alias, null) is KeyStore.PrivateKeyEntry
    } catch (e: Exception) {
        false
    }
}
```

What to do in UserRepository.kt

Modify `ensureKeyPairAndUploadPublicKey()` to detect a missing key and handle re-registration gracefully:

```
// In UserRepository.kt - replace/update ensureKeyPairAndUploadPublicKey()
suspend fun ensureKeyPairAndUploadPublicKey(userId: String): Result<Unit> {
    return try {
        val keyHelper = KeystoreHelper()

        // STEP 1: Check if key still exists in Keystore
        if (!keyHelper.isValidKeyPair()) {
            // Key is gone (reinstall / new device)
            // Generate a brand new keypair
            keyHelper.generateAndStoreRsaKeyPair()

            // Emit a one-time event so the UI can warn the user
            // (see UiEvent below - add this to your ViewModel)
            _uiEvent.emit(UiEvent.KeysRegeneratedWarning)
        }

        // STEP 2: Upload the current public key to Firestore
        val publicKey = keyHelper.getOrCreateRsaKeyPair().public
        val publicKeyBase64 = Base64.encodeToString(
            publicKey.encoded, Base64.NO_WRAP
        )
        firestore.collection("users")
            .document(userId)
            .update("publicKey", publicKeyBase64)
            .await()

        Result.success(Unit)
    } catch (e: Exception) {
        Result.failure(e)
    }
}
```

What to do in ConversationListScreen.kt (or your main ViewModel)

Listen for the `KeysRegeneratedWarning` event and show a one-time Snackbar warning:

```
// In ConversationListViewModel.kt - add to your UiEvent sealed class
sealed class UiEvent {
    object KeysRegeneratedWarning : UiEvent()
    // ... your other events
}
```

```

}

// In ConversationListScreen.kt — inside LaunchedEffect
LaunchedEffect(Unit) {
    viewModel.uiEvent.collect { event ->
        when (event) {
            is UiEvent.KeysRegeneratedWarning -> {
                snackbarHostState.showSnackbar(
                    message = "Security keys were reset. Previous messages cannot
be decrypted.",
                    duration = SnackbarDuration.Long
                )
            }
        }
    }
}
}

```

BUG FIX #2 — Silent Encryption Failure When Recipient Key Is Missing [SEVERITY: CRITICAL]

Problem

In `MessageRepository.sendMessage()`, the code fetches the recipient's public key from Firestore and then calls `encrypt()`. If the recipient's key field is null, empty, or the Firestore read fails due to a network error, `encrypt()` will either throw an unhandled exception (crash) or encrypt with a null key (producing garbage that can never be decrypted). In both cases, no error is shown to the user — the message either disappears silently or the app crashes.

What to do in `MessageRepository.kt`

```

// In MessageRepository.kt — replace your sendMessage() function
suspend fun sendMessage(
    conversationId: String,
    senderId: String,
    plaintext: String
): Result<Unit> {
    return try {
        // STEP 1: Fetch recipient's public key with explicit null-check
        val conversationDoc = firestore
            .collection("conversations")
            .document(conversationId)
            .get()
            .await()

        val participants = conversationDoc.get("participants") as? List<*>
        val recipientId = participants?.firstOrNull { it != senderId } as? String
        ?: return Result.failure(Exception("Could not determine recipient"))

        val recipientDoc = firestore
            .collection("users")
            .document(recipientId)
            .get()
            .await()

        val recipientPublicKeyBase64 = recipientDoc.getString("publicKey")
    } catch (e: Exception) {
        return Result.failure(e)
    }
}

```

```

        // STEP 2: Hard-stop if key is missing – NEVER send unencrypted
        if (recipientPublicKeyBase64.isNullOrBlank()) {
            return Result.failure(
                Exception("Recipient has not set up encryption keys yet. Ask them
to open the app.")
            )
        }

        // STEP 3: Decode and encrypt
        val recipientPublicKeyBytes = Base64.decode(
            recipientPublicKeyBase64, Base64.NO_WRAP
        )
        val keyFactory = KeyFactory.getInstance("RSA")
        val recipientPublicKey = keyFactory.generatePublic(
            X509EncodedKeySpec(recipientPublicKeyBytes)
        )

        val encryptedPayload = MessageCrypto.encrypt(plaintext, recipientPublicKey)

        // STEP 4: Save to Firestore
        val messageData = hashMapOf(
            "senderId"      to senderId,
            "ciphertext"    to encryptedPayload.ciphertext,
            "encryptedKey"  to encryptedPayload.encryptedKey,
            "iv"            to encryptedPayload.iv,
            "timestamp"     to FieldValue.serverTimestamp(),
            "isEdited"      to false
        )

        firestore.collection("conversations")
            .document(conversationId)
            .collection("messages")
            .add(messageData)
            .await()

        Result.success(Unit)
    } catch (e: Exception) {
        Result.failure(e)
    }
}

```

What to do in ChatViewModel.kt

Handle the failure Result and show it as a Snackbar — never swallow the error:

```

// In ChatViewModel.kt – update your sendMessage() function
fun sendMessage(conversationId: String, text: String) {
    if (text.isBlank()) return
    viewModelScope.launch {
        val result = messageRepository.sendMessage(
            conversationId = conversationId,
            senderId = currentUserId,
            plaintext = text.trim()
        )
        if (result.isFailure) {
            _uiEvent.emit(
                UiEvent.ShowError(
                    result.exceptionOrNull()?.message
                        ?: "Failed to send message"
                )
            )
        }
    }
}

```

```

    }
}
}

```

BUG FIX #3 — Message Edit Does Not Re-Encrypt (IV + Ciphertext Mismatch) [SEVERITY: HIGH]

Problem

AES-GCM uses a unique random IV (Initialization Vector) for every single encryption operation. When a user edits a message, the current code likely updates the ciphertext field in Firestore without generating a new IV and a new encrypted AES key. This means the stored IV no longer matches the ciphertext, and AES-GCM authentication will fail on every decryption attempt — the recipient sees nothing or the app crashes.

What to do in MessageRepository.kt

Add an editMessage() function that runs the full encryption pipeline from scratch:

```

// In MessageRepository.kt — add this function
suspend fun editMessage(
    conversationId: String,
    messageId: String,
    newPlaintext: String,
    recipientPublicKey: PublicKey    // pass this from ChatViewModel
): Result<Unit> {
    return try {
        // CRITICAL: Re-run full encryption — new AES key + new IV every time
        val encryptedPayload = MessageCrypto.encrypt(newPlaintext,
            recipientPublicKey)

        // Update ALL three encrypted fields atomically
        firestore.collection("conversations")
            .document(conversationId)
            .collection("messages")
            .document(messageId)
            .update(
                "ciphertext",    encryptedPayload.ciphertext,
                "encryptedKey",  encryptedPayload.encryptedKey,
                "iv",            encryptedPayload.iv,                // MUST be updated
                "isEdited",      true,
                "editedAt",      FieldValue.serverTimestamp()
            )
            .await()

        Result.success(Unit)
    } catch (e: Exception) {
        Result.failure(e)
    }
}

// IMPORTANT: Never do this — it breaks decryption:
// .update("ciphertext", newEncryptedText)    // ← WRONG, IV still points to old
// operation
// Always update ciphertext + encryptedKey + iv together in one .update() call

```

SECTION 3 — New Features to Add

Add these features in priority order. Each section includes the exact Firestore schema changes, ViewModel changes, and Compose UI code needed.

FEATURE #1 — Real-Time Typing Indicators

Firestore schema change

Add two fields to the conversations/{conversationId} document:

```
// Firestore: conversations/{conversationId}
{
  "participants": ["uid1", "uid2"],
  "typingUsers": []    // <-- add this: array of UIDs currently typing
}
```

What to do in ChatViewModel.kt

```
// In ChatViewModel.kt
private var typingJob: Job? = null
private val TYPING_TIMEOUT_MS = 3000L

// Call this every time the user types a character in the text field
fun onTextChanged(newText: String, conversationId: String) {
    _messageText.value = newText

    // Write typing indicator
    typingJob?.cancel()
    setTypingIndicator(conversationId, isTyping = true)

    // Auto-clear after 3 seconds of no typing
    typingJob = viewModelScope.launch {
        delay(TYPING_TIMEOUT_MS)
        setTypingIndicator(conversationId, isTyping = false)
    }
}

// Also clear when message is sent
fun sendMessage(conversationId: String, text: String) {
    typingJob?.cancel()
    setTypingIndicator(conversationId, isTyping = false)
    // ... rest of send logic
}

private fun setTypingIndicator(conversationId: String, isTyping: Boolean) {
    viewModelScope.launch {
        val uid = currentUserId
        val ref = firestore.collection("conversations").document(conversationId)
        if (isTyping) {
            ref.update("typingUsers", FieldValue.arrayUnion(uid))
        } else {
            ref.update("typingUsers", FieldValue.arrayRemove(uid))
        }
    }
}
```

```

    }
}

// Observe typing state - collect in ChatScreen
val isRecipientTyping: StateFlow<Boolean> = flow {
    firestore.collection("conversations").document(conversationId)
        .snapshots()
        .collect { snapshot ->
            val typingUsers = snapshot.get("typingUsers") as? List<*> ?:
emptyList<Any>()
            emit(typingUsers.any { it != currentUserId })
        }
}.stateIn(viewModelScope, SharingStarted.WhileSubscribed(), false)

```

What to do in ChatScreen.kt

```

// In ChatScreen.kt - add below the message list, above the text field
val isRecipientTyping by viewModel.isRecipientTyping.collectAsState()

AnimatedVisibility(
    visible = isRecipientTyping,
    enter = fadeIn() + slideInVertically(),
    exit = fadeOut() + slideOutVertically()
) {
    Row(
        modifier = Modifier.padding(horizontal = 16.dp, vertical = 4.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        // Animated dots
        val infiniteTransition = rememberInfiniteTransition(label = "typing")
        (0..2).forEach { i ->
            val alpha by infiniteTransition.animateFloat(
                initialValue = 0.3f, targetValue = 1f,
                animationSpec = infiniteRepeatable(
                    animation = tween(600, delayMillis = i * 200),
                    repeatMode = RepeatMode.Reverse
                ), label = "dot$i"
            )
            Box(
                modifier = Modifier
                    .padding(horizontal = 2.dp)
                    .size(7.dp)
                    .clip(CircleShape)
                    .background(MaterialTheme.colorScheme.onSurface.copy(alpha =
alpha))
            )
        }
        Spacer(modifier = Modifier.width(6.dp))
        Text("typing...", style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurface.copy(alpha = 0.6f))
    }
}

// In your TextField - hook onValueChange to viewModel.onTextChanged:
OutlinedTextField(
    value = messageText,
    onValueChange = { viewModel.onTextChanged(it, conversationId) },
    // ... rest of your existing TextField params
)

```

FEATURE #2 — Biometric App Lock (Fingerprint / Face ID)

Add dependency to app/build.gradle.kts

```
implementation("androidx.biometric:biometric:1.2.0-alpha05")
```

Create BiometricHelper.kt in a new security/ folder

```
// app/src/main/java/com/application/cryptotalk/security/BiometricHelper.kt
class BiometricHelper(private val activity: FragmentActivity) {

    fun isBiometricAvailable(): Boolean {
        val biometricManager = BiometricManager.from(activity)
        return biometricManager.canAuthenticate(
            BiometricManager.Authenticators.BIOMETRIC_STRONG or
            BiometricManager.Authenticators.DEVICE_CREDENTIAL
        ) == BiometricManager.BIOMETRIC_SUCCESS
    }

    fun authenticate(
        onSuccess: () -> Unit,
        onError: (String) -> Unit
    ) {
        val prompt = BiometricPrompt(
            activity,
            ContextCompat.getMainExecutor(activity),
            object : BiometricPrompt.AuthenticationCallback() {
                override fun onAuthenticationSucceeded(result:
BiometricPrompt.AuthenticationResult) {
                    onSuccess()
                }
                override fun onAuthenticationError(errorCode: Int, errString:
CharSequence) {
                    onError(errString.toString())
                }
                override fun onAuthenticationFailed() {
                    onError("Authentication failed. Try again.")
                }
            }
        )

        val promptInfo = BiometricPrompt.PromptInfo.Builder()
            .setTitle("Unlock CryptoTalk")
            .setSubtitle("Use your fingerprint or face to continue")
            .setAllowedAuthenticators(
                BiometricManager.Authenticators.BIOMETRIC_STRONG or
                BiometricManager.Authenticators.DEVICE_CREDENTIAL
            )
            .build()

        prompt.authenticate(promptInfo)
    }
}
```

What to do in MainActivity.kt

Show the biometric prompt on every app resume. Store unlocked state in a ViewModel so it survives recomposition:

```
// In MainActivity.kt
class MainActivity : ComponentActivity() {
    private val authViewModel: AuthLockViewModel by viewModels()

    override fun onResume() {
        super.onResume()
        // Re-lock every time app comes to foreground
        if (BiometricHelper(this).isBiometricAvailable()) {
            authViewModel.lock()
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            CryptoTalkTheme {
                val isUnlocked by authViewModel.isUnlocked.collectAsState()

                if (!isUnlocked) {
                    BiometricLockScreen(
                        onUnlock = {
                            BiometricHelper(this).authenticate(
                                onSuccess = { authViewModel.unlock() },
                                onError = { /* show error, keep locked */ }
                            )
                        }
                    )
                } else {
                    CryptoTalkNavHost(/* your existing params */)
                }
            }
        }
    }
}

// AuthLockViewModel.kt
class AuthLockViewModel : ViewModel() {
    private val _isUnlocked = MutableStateFlow(false)
    val isUnlocked: StateFlow<Boolean> = _isUnlocked.asStateFlow()
    fun unlock() { _isUnlocked.value = true }
    fun lock() { _isUnlocked.value = false }
}

// BiometricLockScreen.kt (simple Compose screen)
@Composable
fun BiometricLockScreen(onUnlock: () -> Unit) {
    LaunchedEffect(Unit) { onUnlock() } // Trigger prompt immediately on show
    Box(modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Icon(Icons.Default.Lock, contentDescription = null,
                modifier = Modifier.size(64.dp),
                tint = MaterialTheme.colorScheme.primary)
            Spacer(modifier.height(16.dp))
            Text("CryptoTalk is locked", style =
MaterialTheme.typography.titleMedium)
            Spacer(modifier.height(8.dp))
            TextButton(onClick = onUnlock) { Text("Unlock") }
        }
    }
}
```

```
}
```

FEATURE #3 — Self-Destructing Messages (Auto-Delete After Read)

Firestore schema change

```
// Add these optional fields to each message document
{
  "senderId":      "uid1",
  "ciphertext":    "...",
  "encryptedKey":  "...",
  "iv":            "...",
  "timestamp":     Timestamp,
  "isEdited":      false,
  "selfDestruct":  true,           // <-- new: whether this message self-destructs
  "destructAfterMs": 10000        // <-- new: milliseconds after read (e.g. 10
seconds)
}
```

Add a DeleteAfterRead UI toggle in ChatScreen.kt

```
// In ChatScreen.kt — add a toggle button next to the send button
var selfDestructEnabled by remember { mutableStateOf(false) }
var selectedDuration by remember { mutableStateOf(10_000L) } // 10 seconds default

IconToggleButton(
    checked = selfDestructEnabled,
    onCheckedChange = { selfDestructEnabled = it }
) {
    Icon(
        if (selfDestructEnabled) Icons.Default.Timer else Icons.Default.TimerOff,
        contentDescription = "Self-destruct toggle",
        tint = if (selfDestructEnabled) MaterialTheme.colorScheme.error
              else MaterialTheme.colorScheme.onSurface
    )
}

// Pass selfDestruct params to ViewModel when sending:
viewModel.sendMessage(
    conversationId = conversationId,
    text = messageText,
    selfDestruct = selfDestructEnabled,
    destructAfterMs = selectedDuration
)
```

Auto-delete logic in ChatViewModel.kt

```
// In ChatViewModel.kt — call this when a self-destruct message is displayed
fun scheduleMessageDeletion(conversationId: String, messageId: String, delayMs:
Long) {
    viewModelScope.launch {
        delay(delayMs)
        firestore.collection("conversations")
            .document(conversationId)
            .collection("messages")
            .document(messageId)
```

```

        .delete()
        .await()
    }
}

// In your message rendering – detect and schedule deletion when message is shown
// In the LazyColumn items block in ChatScreen.kt:
if (message.selfDestruct && message.senderId != currentUserId) {
    LaunchedEffect(message.id) {
        viewModel.scheduleMessageDeletion(
            conversationId, message.id, message.destructAfterMs
        )
    }
    // Show a countdown timer in the message bubble
    SelfDestructCountdown(destructAfterMs = message.destructAfterMs)
}

```

FEATURE #4 — Message Read Receipts (Single / Double Tick)

Firestore schema change

```

// Add to each message document:
{
    // ... existing fields
    "status": "sent" // values: "sent" | "delivered" | "read"
}

```

What to do in MessageRepository.kt

```

// Add this function to mark messages as read
suspend fun markMessagesAsRead(conversationId: String, currentUserId: String) {
    val unreadMessages = firestore
        .collection("conversations")
        .document(conversationId)
        .collection("messages")
        .whereNotEqualTo("senderId", currentUserId)
        .whereEqualTo("status", "sent")
        .get()
        .await()

    val batch = firestore.batch()
    unreadMessages.documents.forEach { doc ->
        batch.update(doc.reference, "status", "read")
    }
    batch.commit().await()
}

// In sendMessage() – set initial status to "sent":
val messageData = hashMapOf(
    // ... existing fields
    "status" to "sent" // <-- add this line
)

```

Tick icons in ChatScreen.kt

```

// Add this composable for the tick indicator

```

```

@Composable
fun MessageStatusIcon(status: String, isSentByMe: Boolean) {
    if (!isSentByMe) return // Only show ticks on outgoing messages
    val (icon, tint) = when (status) {
        "read" -> Icons.Default.DoneAll to MaterialTheme.colorScheme.primary
        "delivered" -> Icons.Default.DoneAll to
        MaterialTheme.colorScheme.onSurface.copy(alpha = 0.5f)
        else -> Icons.Default.Done to
        MaterialTheme.colorScheme.onSurface.copy(alpha = 0.5f)
    }
    Icon(icon, contentDescription = status,
        modifier = Modifier.size(14.dp).padding(start = 2.dp), tint = tint)
}

// In ChatScreen.kt - call markMessagesAsRead when chat opens:
LaunchedEffect(conversationId) {
    viewModel.markMessagesAsRead(conversationId)
}

```

FEATURE #5 — Push Notifications via Firebase Cloud Messaging (FCM)

Add dependency to app/build.gradle.kts

```

implementation(platform("com.google.firebase:firebase-bom:33.0.0"))
implementation("com.google.firebase:firebase-messaging-ktx")

```

Save FCM token to Firestore on login — add to UserRepository.kt

```

// In UserRepository.kt
suspend fun saveFcmToken(userId: String) {
    FirebaseMessaging.getInstance().token.await().let { token ->
        firestore.collection("users")
            .document(userId)
            .update("fcmToken", token)
            .await()
    }
}

// Call saveFcmToken() from AuthRepository after successful sign-in

```

Create CryptoTalkMessagingService.kt

```

// app/src/main/java/com/application/cryptotalk/CryptoTalkMessagingService.kt
class CryptoTalkMessagingService : FirebaseMessagingService() {

    override fun onMessageReceived(remoteMessage: RemoteMessage) {
        // IMPORTANT: The notification payload NEVER contains message content
        // It only tells the receiver a new encrypted message is waiting
        val senderName = remoteMessage.data["senderName"] ?: "Someone"

        val notification = NotificationCompat.Builder(this, "messages_channel")
            .setSmallIcon(R.drawable.ic_launcher_foreground)
            .setContentTitle("CryptoTalk")
            .setContentText("New message from $senderName") // Never show message
content
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setAutoCancel(true)
    }
}

```

```

        .build()

        NotificationManagerCompat.from(this)
            .notify(System.currentTimeMillis().toInt(), notification)
    }

    override fun onNewToken(token: String) {
        // Token refreshed – update in Firestore
        val userId = FirebaseAuth.getInstance().currentUser?.uid ?: return
        CoroutineScope(Dispatchers.IO).launch {
            FirebaseFirestore.getInstance()
                .collection("users")
                .document(userId)
                .update("fcmToken", token)
                .await()
        }
    }
}

// Register in AndroidManifest.xml inside <application>:
// <service android:name=".CryptoTalkMessagingService"
//         android:exported="false">
//     <intent-filter>
//         <action android:name="com.google.firebase.MESSAGING_EVENT"/>
//     </intent-filter>
// </service>

```

SECTION 4 — Cleanup & Security Hardening

REMOVE — WebViewActivity.kt

Delete `WebViewActivity.kt` from the `ui/` folder and remove its declaration from `AndroidManifest.xml`. Search the entire codebase for any `Intent` or `NavRoutes` reference to `WebView` and remove those too. If you need to open a URL (e.g., a privacy policy link), use this safe replacement instead:

```

// Safe way to open a URL – use CustomTabsIntent, never WebView
val customTabIntent = CustomTabsIntent.Builder()
    .setShowTitle(true)
    .build()
customTabIntent.launchUrl(context, Uri.parse("https://your-privacy-policy-url.com"))

```

ENABLE — ProGuard / R8 Code Obfuscation for Release Builds

In `app/build.gradle.kts`, update your release build type to enable shrinking and obfuscation:

```

// In app/build.gradle.kts
android {
    buildTypes {
        release {
            isMinifyEnabled = true           // Enable R8 shrinking + obfuscation
            isShrinkResources = true         // Remove unused resources
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
            )
        }
    }
}

```

Add these rules to `app/proguard-rules.pro` to protect your crypto classes while keeping Firebase working:

```
# Keep Firebase
-keep class com.google.firebase.** { *; }
-keep class com.google.android.gms.** { *; }

# Keep your crypto classes (obfuscating these can break key operations)
-keep class com.application.cryptotalk.crypto.** { *; }

# Keep data model classes (Firestore serialization needs field names)
-keep class com.application.cryptotalk.data.model.** { *; }
```

REMOVE — Cleartext Metadata in Firestore

Review your Firestore message documents. Remove any fields that store the sender name, receiver name, or message preview as plain readable text. The only safe metadata fields are:

- senderId (UID, not email or display name)
- timestamp (server timestamp — unavoidable for ordering)
- status (sent/delivered/read)
- isEdited, editedAt
- selfDestruct, destructAfterMs (if using Feature #3)

Do NOT store: senderName, recipientEmail, messagePreview, or any other human-readable identifiers in cleartext on the server.

SECTION 5 — Testing Checklist After Implementation

After completing the above changes, verify each scenario manually:

#	Test scenario	Expected result
1	Clear app data / reinstall. Log in with same Google account.	Warning Snackbar shown. New public key uploaded. App does not crash.
2	User A sends message to User B whose account exists but never opened the app (no key).	Snackbar: 'Recipient has not set up keys.' Message NOT sent. App does not crash.
3	User A sends a message. User A edits it. User B reads the edited version.	User B sees the edited text correctly decrypted. No authentication error.
4	Open app. Trigger biometric prompt. Cancel / fail biometric.	App stays on lock screen. Does not navigate to chat.
5	User A starts typing. User B's screen shows '...' typing indicator within 1-2 seconds.	Typing dots appear, then disappear 3 seconds after A stops typing.
6	Send a self-destruct message with 10-second timer.	Message disappears from both devices after 10 seconds of being read.

7	User B reads all messages in a conversation.	User A sees double-blue ticks on all messages.
8	Enable ProGuard and build a release APK. Install and run.	App works normally. Crypto functions not broken by obfuscation.